

应用4：背包问题

问题

- 有个小偷，光顾一家超级市场，他带了一个背包来装所偷的东西，假设他的背包最多只能装三十公斤，而超市内的每样东西有它的重量及价值，小偷背包问题是要找出最佳的偷法，使得背包内所装的赃物总价值最高，且总重量又不超过三十公斤。这样的一个问题居然也是难题！如果小偷用数字计算器来替他决定最好的偷法，在他得到答案前，可能早就被绳之以法了。

问题

- 给定一个载重量为 M 的背包， n 个重量为 w_i ，价值为 p_i 的物体， $1 \leq i \leq n$ ，要求把物体装入背包，使背包内的物体价值最大，把这类问题称为背包问题。

问题

$Knap(l, n, c)$ 定义如下：

$$\begin{aligned} & \max \sum_{i=l}^n v_i x_i && v_i > 0 \\ & \left\{ \begin{array}{ll} \sum_{i=l}^n w_i x_i \leq c & w_i > 0 \\ x_i \in \{0,1\} & l \leq i \leq n \end{array} \right. && \text{求}(x_l, x_{l+1}, \dots, x_n) \text{使目标函数最大} \end{aligned}$$

举例： **$w=(w_1, w_2, w_3)=(2, 3, 4)$, $v=(v_1, v_2, v_3)=(1, 2, 5)$, $n=3$, $c=6$, 求 $Knap(1, 3, 6)$ 。取 $x=(1, 0, 1)$ 时，**

$$\sum_{i=1}^n v_i x_i = 1 \cdot 1 + 2 \cdot 0 + 5 \cdot 1 = 6 \text{ 最大}$$

0-1背包问题([王]p71)

- 输入: n 物品重 $W[1:n]$, 价值 $V[1:n]$, 背包容量 C
- 输出: 装包使得价值最大.
- 例: $W:\{2,2,6,5,4\}$
 $V:\{6,3,5,4,6\}$
 $C=10$

求解(1): 目标函数

- 假设: x_i 表示物品的被装入背包的情况, $x_i=0,1$ 。当 $x_i=1$,表示物体被装入背包。根据问题的要求, 我们有约束方程和目标函数:

- $$\sum_{i=1}^n w_i x_i \leq M \quad \text{optp} = \max \sum_{i=1}^n p_i x_i$$

问题归结为寻找一个满足约束方程, 并使目标函数达到最大的解向量 $X=(x_1, x_2, \dots, x_n)$ 。

求解(1): 目标函数

- 如何分解问题?
 - 第一阶段: 装入一个物体, 求解在不同载重量的背包下, 能够得到的最大价值;
 - 第二阶段: 装入前两个物体, 求解在不同载重量的背包下, 能够得到的最大价值;
 - 以此类推;
 - 最后第 n 个阶段, 装入 n 个物品, 求解在不同载重量的背包下, 能够取得的最大价值。

求解(2): 递归函数

- 令 $\text{opt}q_i(j)$ 表示在前 i 个物体中, 能够装入载重量为 j 的背包中的物体的最大价值。
- 如何构建递归函数?
- 0-1背包问题是否满足最优化原理?

求解(2): 递归函数

- 令 $\text{optq}_i(j)$ 表示在前 i 个物体中, 能够装入载重量为 j 的背包中的物体的最大价值, $j=1,2,\dots,m$ 。可以得到下面的动态规划函数:

$$\text{optp}_i(0) = \text{optp}_0(j) = 0$$

$$\text{optp}_i(j) = \begin{cases} \text{optp}_{i-1}(j) & j < w_i \\ \max\{\text{optp}_{i-1}(j), \text{optp}_{i-1}(j - w_i) + p_i\} & j > w_i \end{cases}$$

0-1背包问题OSP

$$\max \sum_{i=l}^n v_i x_i$$

最优子结构性质证明

$$\begin{cases} \sum_{i=l}^n w_i x_i \leq c \\ x_i \in \{0,1\} \quad l \leq i \leq n \end{cases} \xRightarrow{\text{记}} \text{Knap}(l, n, c)$$

- 证明：设 $(y_1, y_2, \dots, y_n)$ 是 $\text{Knap}(1, n, c)$ 的一个最优解，则
- $(y_2, \dots, y_n)$ 是 $\text{Knap}(2, n, c - w_1 y_1)$ 子问题的一个最优解。
- 若不然，设 $(z_2, \dots, z_n)$ 是 $\text{Knap}(2, n, c - w_1 y_1)$ 的最优解，因此有

$$\sum_{i=2}^n v_i z_i > \sum_{i=2}^n v_i y_i \quad \text{且} \quad \sum_{i=2}^n w_i z_i \leq c - w_1 y_1$$

$$\Rightarrow v_1 y_1 + \sum_{i=2}^n v_i z_i > \sum_{i=1}^n v_i y_i \quad \text{又有} \quad w_1 y_1 + \sum_{i=2}^n w_i z_i \leq c$$

说明 $(y_1, z_2, \dots, z_n)$ 是 $\text{Knap}(1, n, c)$ 的一个更优解，矛盾。

0-1背包问题及(分数)背包OSP

此问题的形式化描述是,给定 $c > 0, w_i > 0, v_i > 0, 1 \leq i \leq n$, 要求找出一个 n 元向量 $(x_1, x_2, \dots, x_n), 0 \leq x_i \leq 1, 1 \leq i \leq n$, 使得 $\sum_{i=1}^n w_i x_i \leq c$, 而且 $\sum_{i=1}^n v_i x_i$ 达到最大。

这两类问题都具有最优子结构性质。对于0-1背包问题,设 A 是能够装入容量为 c 的背包的具有最大价值的物品集合,则 $A_j = A - \{j\}$ 是 $n-1$ 个物品 $1, 2, \dots, j-1, j+1, \dots, n$ 可装入容量为 $c - w_j$ 的背包的具有最大价值的物品集合。对于背包问题,类似地,若它的一个最优解包含物品 j ,则从该最优解中拿出所含的物品 j 的那部分重量 w , 剩余的将是 $n-1$ 个原重物品 $1, 2, \dots, j-1, j+1, \dots, n$ 以及重为 $w_j - w$ 的物品 j 中可装入容量为 $c - w$ 的背包且具有最大价值的物品。

求解(3): 递归求解最优值

- 第一阶段: 装入一个物体, 求解在不同载重量的背包下, 能够得到的最大价值;
- 第二阶段: 装入前两个物体, 求解在不同载重量的背包下, 能够得到的最大价值;
- 以此类推;
- 最后第 n 个阶段, $optp_n(m)$ 是在载重量为 m 的背包下, 装入 n 个物品, 能够取得的最大价值。

求解(4): 构造最优解

- 确定装入背包的具体物体, 从 $\text{optp}_n(m)$ 的值向前推倒。由此得到递推关系式:
- 若 $\text{optp}_i(j)=\text{optp}_{i-1}(j)$, 则 $x_i=0$
- 若 $\text{optp}_i(j)>\text{optp}_{i-1}(j)$, 则 $x_i=1$,
 $j=j-w_i$ 。

0-1背包问题

例：有5个物体，重量2,2,6,5,4，价值6,3,5,4,6，背包的载重量为10，求装入背包的物体及其总价值

	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	6	6	<u>6</u>	6	6	6	6	6	6
2	0	0	6	6	9	9	<u>9</u>	9	9	9	9
3	0	0	6	6	9	9	9	9	11	11	14
4	0	0	6	6	9	9	9	10	11	13	14
5	0	0	6	6	9	9	12	12	15	15	<u>15</u>

$$\text{optp}_i(0) = \text{optp}_0(j) = 0$$

$$\text{optp}_i(j) = \begin{cases} \text{optp}_{i-1}(j) & j < w_i \\ \max\{\text{optp}_{i-1}(j), \text{optp}_{i-1}(j - w_i) + p_i\} & j > w_i \end{cases}$$

装入背包的物体的最大价值为15，装入背包的物体为
 $X = \{1, 1, 0, 0, 1\}$ 。

0-1背包: 编程

- 输入: n 物品重 $W[1:n]$, 价值 $V[1:n]$, 背包容量 C
- 输出: 装包使得价值最大 (物品重量为**整数**).
- $[1:i]$, $dp[i,k]$ = 由 $[1:i]$ 组合出重量 $\leq k$ 的最大价值
- $dp[i,k] = \max\{ dp[i-1,k], dp[i-1,k-W[i]] + V[i] \}$ (OSP)
- 最优值: $dp[n,C]$
 1. 初始 $dp[0,0:C] = 0$,
 2. 对 $i = 1:n$, 对 $k = 1:C$,
 3. $dp[i,k] = dp[i-1,k]$
 4. 若 $k \geq W[i]$, $pt = dp[i-1,k-W[i]] + V[i]$;
 5. 若 $pt > dp[i,k]$, $dp[i,k] = pt$
 6. 输出 $dp[n,C]$ //时间 $O(nC)$.

0-1背包问题推广 找零钱问题

一出纳员支付一定数量的现金. 假设他手中有几种面值的硬币, 要求他用最少的硬币数支付规定的现金.

例如, 现有3种硬币: 它们的面值分别为1元、4元和6元. 要支付8元.

0-1背包问题推广 找零钱问题

$p_i(j)$: 前 i 种硬币支付金额 j 所需硬币的最少个数。

$$p_i(j) = \begin{cases} p_{i-1}(j) & j < v_i \\ \min\{p_{i-1}(j), p_i(j - v_i) + 1\} & j \geq v_i \end{cases}$$

不用第 i 种硬币

用第 i 种硬币

$$p_i(0) = 0$$

$$p_0(j) = \infty$$

找零钱问题：1元、4元和6元. 要支付8元.

$$p_i(j) = \begin{cases} p_{i-1}(j) & j < v_i \\ \min\{p_{i-1}(j), p_{\textcolor{red}{i}}(j - v_i) + \textcolor{red}{1}\} & j \geq v_i \end{cases}$$

	0	1	2	3	4	5	6	7	8
0	0	∞	∞	∞	∞	∞	∞	∞	∞
1	0	1	2	3	4	5	6	7	8
2	0	1	2	3	1	2	3	4	2
3	0	1	2	3	1	2	1	2	2

(分数)背包问题(knapsack Prob)

0-1背包问题

- 输入: n 物品重 $W[1:n]$,
价值 $V[1:n]$,
背包容量 C
- 物品重量为**整数**.
- 输出: 装包使得价值最大.
- 每件物品只能取或不取

$$\max \sum_{i=1}^n V_i x_i$$

$$\sum_{i=1}^n W_i x_i \leq C$$

$$x_i \in \{0,1\}, 1 \leq i \leq n$$

(分数)背包问题

- 输入: 物品重 $W[1:n]$, 价值 $V[1:n]$
- 输出: 装包使得价值最大.
- 物品 i 可以取重量 $0 \sim W[i]$

$$\max \sum_{i=1}^n V_i x_i$$

$$\sum_{i=1}^n W_i x_i \leq C$$

$$0 \leq x_i \leq 1, 1 \leq i \leq n$$

(分数)背包问题

(分数)背包问题

- 输入: 物品重 $W[1:n]$, 价值 $V[1:n]$
- 输出: 装包使得价值最大.
- 物品 i 可以取重量 $0 \sim W[i]$

$$\max \sum_{i=1}^n V_i x_i$$

$$\sum_{i=1}^n W_i x_i \leq C$$

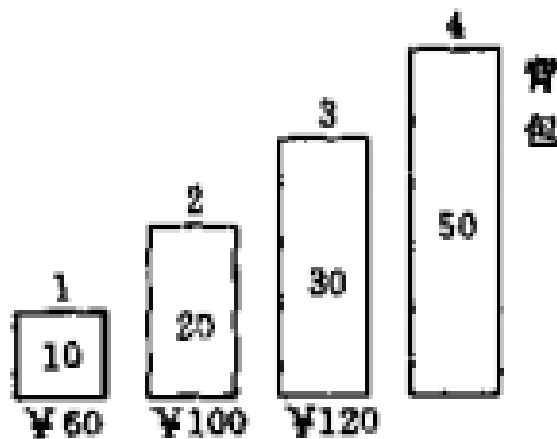
$$0 \leq x_i \leq 1, 1 \leq i \leq n$$

贪心算法:

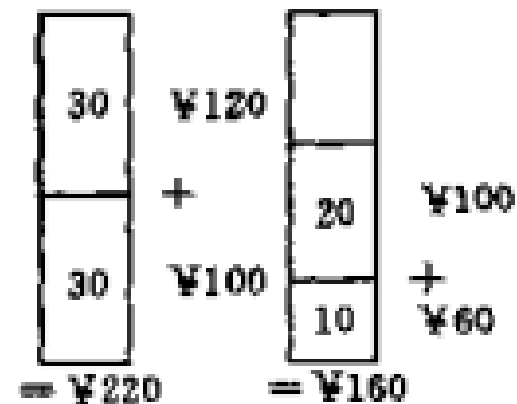
1. 对单位重量价值 $\{ V[i]/W[i] \}_{i=1}^n$ 排序
 2. 优先放入单位重量价值大的物品, 直到填满
- $O(n \log n)$,
- 0-1背包能贪心吗?

(分数)背包问题

0-1背包能贪心吗？



(a)



(b)

最优装载([王]p95)

0-1背包问题

- 输入: n 物品重 $W[1:n]$,
价值 $V[1:n]$,
背包容量 C
- 物品重量为整数.
- 输出: 装包使得价值最大.
- 每件物品只能取或不取

$$\max \sum_{i=1}^n V_i x_i$$

$$\sum_{i=1}^n W_i x_i \leq C$$

$$x_i \in \{0,1\}, 1 \leq i \leq n$$

最优装载

- 输入: n 物品重 $W[1:n]$,
背包容量 C
- 输出: 装包使得件数最多.
- 每件物品只能取或不取

$$\max \sum_{i=1}^n x_i$$

$$\sum_{i=1}^n W_i x_i \leq C$$

$$x_i \in \{0,1\}, 1 \leq i \leq n$$

最优装载

最优装载

- 输入: n 物品重 $W[1:n]$,
背包容量 C
- 输出: 装包使得件数最多.
- 每件物品只能取或不取

$$\max \sum_{i=1}^n x_i$$

$$\sum_{i=1}^n W_i x_i \leq C$$

$$x_i \in \{0,1\}, 1 \leq i \leq n$$

贪心算法($O(n \log n)$):

1. 对重量 $W[1:n]$ 升序排列
2. 优先添加重量小的物品, 直到不能再添加

贪心选择性质: 最优解可以包含重量最小的

OSP: 最优解($[1:n], C$)去掉重量最小($[2:n], C - W[1]$)仍是最优解

问题

- 0-1背包问题的动态规划方法是采用正向思维还是逆向思维方法？
- 若采用逆向思维方法，如何构建0-1背包问题的动态规划方程？

0-1背包问题

2. 递归关系

设所给 0-1 背包问题的子问题

$$\max \sum_{k=i}^n v_k x_k \quad \begin{cases} \sum_{k=i}^n w_k x_k \leq j \\ x_k \in \{0,1\} \quad i \leq k \leq n \end{cases}$$

的最优值为 $m(i, j)$, 即 $m(i, j)$ 是背包容量为 j , 可选择物品为 $i, i+1, \dots, n$ 时 0-1 背包问题的最优值。由 0-1 背包问题的最优子结构性质, 可以建立计算 $m(i, j)$ 的递归式如下:

$$m(i, j) = \begin{cases} \max \{m(i+1, j), m(i+1, j-w_i) + v_i\} & j \leq w_i \\ m(i+1, j) & 0 \leq j < w_i \end{cases}$$

$$m(n, j) = \begin{cases} v_n & j \geq w_n \\ 0 & 0 \leq j < w_n \end{cases}$$

应用5：资源分配问题

0-1背包问题推广 资源分配问题

某公司准备将4名销售员分配到3个销售点甲乙丙。各销售点增加若干名销售员后可增加的月销售额如下表：

增加销售额（万元）	增1人	增2人	增3人	增4人
甲	12	22	30	38
乙	11	20	24	30
丙	13	25	30	36

公司每月最多可以增加销售额？ 万元

资源分配问题

- 问题描述
- 问题满足最优性原理
- 递归关系推导
- 算法
- 时间分析

问题

- 8万元投资到A，B，C三个项目中，利润表如下，问：如何投资可以获得最大利润？

X	0	1	2	3	4	5	6	7	8
$g_1(x)$	0	4	26	40	45	50	51	52	53
$g_2(x)$	0	5	15	40	60	70	73	74	75
$g_3(x)$	0	5	15	40	80	90	95	98	100

资源分配问题



- 问题描述：有资源 a 分配给 n 个项目， $g_i(x)$ 为将数量为 x 的资源分配给项目 i 的所能得到的利润。
- 求 $\max Z = g_1(x_1) + g_2(x_2) + \dots + \dots + g_n(x_n)$
- 其中： $x_1 + x_2 + \dots + \dots + x_n = a$,
- $x_i \geq 0, i=1, 2, \dots, n$

分析



- 若 $g_i(x_i)$ 是 x_i 的线性函数，则是一般的线性规划问题，可转化为阶段判决问题。
- 设 $f_k(n)$ 为资源 n 分配给前 k 个项目所得的最大利润。设 $d_k(n)$ 表示使 $f_k(n)$ 最大时，分配给第 k 个工程的资源数。
- 利用正向思维法，先求一个变量的最优分配，再一个个增加变量

问题

- 8万元投资到A, B, C三个项目中, 利润表如下, 求最大利润?

X	0	1	2	3	4	5	6	7	8
$g_1(x)$	0	4	26	40	45	50	51	52	53
$g_2(x)$	0	5	15	40	60	70	73	74	75
$g_3(x)$	0	5	15	40	80	90	95	98	100

问题分析

- 第1阶段: 把不同资源分配给第1个工程
- 第2阶段: 把不同资源分配给前2个工程
- 第3阶段: 把不同资源分配给前3个工程
-
- 第 n 阶段: 把不同资源分配给前 n 个工程

分析(续)

- 初始状态:
 - $f_1(a) = \max_{0 \leq x \leq a} g_1(x)$
-
- 子问题的递归函数

当 $x=2$ 万元时, $f_2(2) = \max\{g_2(0)+f_1(2), g_2(1)+f_1(1), g_2(2)+f_1(0)\}$

- $= \max\{26, 9, 15\} = 26$

分析(续)

- 初始状态:
 - $f_1(a) = \max g_1(x)$
 $0 \leq x \leq a$
- 子问题的递归式(中间状态转换函数):
 - $f_i(a) = \max \{ g_i(x) + f_{i-1}(a-x) \}$
 $0 \leq x \leq a, 2 \leq i \leq n$

计算

- 第1步：计算各个阶段不同分配资源时的最大利润，以及各个工程在此利润下的分配资源数。
- （1）第一阶段：把资源分配给第一个工程
 - » $f_1(1)=4$, $f_1(2)=26$,
 - » $f_1(3)=40$, $f_1(4)=45$,
 - » $f_1(5)=50$, $f_1(6)=51$,
 - » $f_1(7)=52$, $f_1(8)=53$

计算(续)

(2) 第二阶段: 把资源x分配给前二个工程

当x=1万元时, $f_2(1)=\max\{G_2(0)+f_1(1), G_2(1)+f_1(0)\}$

$$\gg =\max\{4,5\}=5$$

$$\gg d_2(1)=1$$

当x=2万元时, $f_2(2)=\max\{G_2(0)+f_1(2), G_2(1)+f_1(1), G_2(2)+f_1(0)\}$

- $=\max\{26, 9, 15\}=26$

- $d_2(2)=0$

计算(续)

类似地计算 $x=3,4,5,6,7,8$ 时的 $f_2(x)$ 的值。

X	0	1	2	3	4	5	6	7	8
$f_2(x)$	0	5	26	40	60	70	86	100	110
$d_2(x)$	0	1	0	0	4	5	4	4	5

同样计算 $f_3(x)$ 的值

X	0	1	2	3	4	5	6	7	8
$f_3(x)$	0	5	26	40	80	90	106	120	140
$d_3(x)$	0	1	0	0	4	5	4	4	4

计算(续)

第2步：求各个阶段的最大利润，以及在此利润下的分配份额；

$$g_i = \max\{f_i(1), \dots, f_i(m)\},$$

$$q_i = \text{使 } f_i(x) \text{ 达最大的 } x$$

$$g_1 = 53, g_2 = 110, g_3 = 140$$

$$q_1 = 8, q_2 = 8, q_3 = 8$$

计算(续)

第3步：计算全局的最大利润 optg ，最大的工程数目 k 以及总的最优分配份额 optx ，有：

$$\text{optg} = 140$$

$$k = 3$$

$$\text{optx} = 8$$

$$\text{optg} = \max\{g_1, g_2, g_3\}$$

$$k = \text{使 } g_i \text{ 最大的 } i$$

$$\text{optx} = \text{与最大的 } g_i \text{ 对应的 } q_i$$

问题

- 请问分配给项目1,2,3的资源是多少？

计算(续)

- 第4步：计算各个工程的最优分配份额；
 - $\text{opt}q_i = d_i(\text{opt}x)$
 - $\text{opt}x = \text{opt}x - \text{opt}q_i \quad i=k, k-1, \dots, 1$

$$\text{opt}q_3 = d_3(\text{opt}x) = d_3(8) = 4 \quad \text{opt}x = \text{opt}x - \text{opt}q_3 = 8 - 4 = 4$$

$$\text{opt}q_2 = d_2(\text{opt}x) = d_2(4) = 4 \quad \text{opt}x = \text{opt}x - \text{opt}q_2 = 4 - 4 = 0$$

$$\text{opt}q_1 = d_1(\text{opt}x) = d_1(0) = 0$$

结果

- 分配给第2，3工程各4万元，可得最大利润140万元。